DAY 5 · SESSION 1 – KUBECTL: DRIVING YOUR CLUSTER FROM THE TERMINAL

Yesterday you clicked it into being. Today you type it.

Yesterday you deployed to the shared ACK cluster by clicking through the Alibaba console. Today you drive it from your own terminal with kubectl – the same commands real teams use every day. And this time, your badge gets a real subdomain of its own, with a real TLS certificate, instead of a shared cluster address.

IN PARTNERSHIP WITH عزم السعودية SAUDI AZM SCCC by stc

THE SESSION · FOUR IDEAS

From clicking to typing, then a real name of your own

IDEA 1

kubectl

The terminal tool that talks to the cluster – everything you clicked, it can type.

`kubeconfig` `context`

IDEA 2

Bare Pods

`kubectl run` – one Pod, no Deployment, no self-healing.

`run` `delete`

IDEA 3

Typed objects

Deployment and Service, same as yesterday – now from `create` / `expose`.

`Deployment` `Service`

IDEA 4

Annotations

Metadata other tools read – this is how your badge gets a real domain.

`external-dns`

> All four ideas build on each other in order – keep this strip in mind as we go.

kubectl: everything you clicked yesterday, now typed

`kubectl` is the command-line tool that talks to a Kubernetes cluster. It knows which cluster and namespace to talk to from a kubeconfig file — your instructor hands you yours, the same way you got an SSH key on Day 1.

```
kubectl config get-contexts          # confirms your kubeconfig loaded correctly
kubectl get pods -n <your-namespace> # same Pods from yesterday, new window
```

Same namespace as Day 4

Every command below is scoped with `-n <your-namespace>` — the same one you created yesterday. You're still walled off from everyone else on the shared cluster.

Nothing to install on the cluster

kubectl is a client you run locally. The cluster itself is already there — you're just pointing a new tool at it.

Pods the hard way: `kubectl run`

```
kubectl run scratch --image=nginx --port=80 -n <your-namespace>
kubectl get pods -n <your-namespace>
kubectl describe pod scratch -n <your-namespace>   # events, status, why it's (not) running
kubectl logs scratch -n <your-namespace>           # what the container printed
kubectl delete pod scratch -n <your-namespace>      # gone — and nothing replaces it
```

No Deployment, no self-healing

That last line is the point: delete a bare Pod and it's just gone. Nothing watches it, nothing replaces it.

Exactly why Day 4 had Deployments

This is the gap a Deployment fills — you're about to create one yourself, typed instead of clicked.

Deployments & Services, from the CLI

```
kubectl create deployment badge --image=<your-dockerhub-username>/cloud-explorer:latest -n <your-namespace>
kubectl expose deployment badge --port=80 --target-port=80 -n <your-namespace>
```

> Notice `kubectl expose` here makes a plain internal address (`ClusterIP`), not a public one – deliberate. Today's badge Deployment sits behind a front door instead of facing the internet directly. Next slide explains why.

VERB	DOES
`kubectl run`	one bare Pod, no self-healing
`kubectl create deployment`	a Deployment – Pods that get replaced if they die
`kubectl expose`	a Service – one stable address in front of the Deployment's Pods
`get / describe / logs`	look, look closer, read what it printed
`kubectl delete`	remove it

One Service holds the domain. One Deployment scales freely.

```
Browser/hey → https://<name>.alialjaffer.com
                    |  (external-dns watches this Service's annotation,
                    |   writes the DNS record — Cloudflare set DNS-only
                    ▼   so hey hits the real origin directly)
   Service: caddy-front-door  (type: LoadBalancer)
                    ▼
   Deployment: caddy-front-door  (replicas: 1, PINNED — never autoscaled)
     · your instructor's own Caddy image · gets the ONE cert for this hostname
                    |  reverse_proxy, plain HTTP, internal only
                    ▼
   Service: badge  (type: ClusterIP, no external IP)
                    ▼
   Deployment: badge  ← THIS is what the HPA scales next session
     · your cloud-explorer image · never touches TLS/DNS · scales freely
```

> A naive "run Caddy in every replica" design breaks under autoscaling: N replicas means N Let's Encrypt requests for the **same** hostname. Splitting front door from backend fixes it.

Annotations vs labels — and the one that gives you a real domain

```
metadata:
  annotations:
    external-dns.alpha.kubernetes.io/hostname: <your-name>.alialjaffer.com
```

Label KUBERNETES READS IT

Metadata Kubernetes itself uses to match objects — how a Service's `selector` finds the right Pods.

Annotation OTHER TOOLS READ IT

Metadata for everything **else**. Kubernetes core doesn't act on it — a controller you installed does.

> external-dns is already installed on the shared cluster. It watches every Service for this one annotation key, and writes the matching DNS record automatically — no console clicking, no manual DNS entry. 🔒 Records are DNS-only (no Cloudflare proxying), so the real IP is always reachable directly.

🏆 MILESTONE 9 · GIVE YOUR BADGE ITS OWN NAME

A real subdomain, a real certificate, all from your terminal

1. Recon yesterday's objects: `kubectl get pods,deployments,services -n <your-namespace>`.
2. Feel the gap. Run a throwaway Pod with `kubectl run`, delete it, watch nothing replace it.
3. Clean slate. Delete yesterday's `hello` Deployment and Service – you won't need them anymore.
4. Rebuild badge as the backend. Download and `apply` it, then `expose` it.
5. Stand up the front door. Fill in your subdomain, `apply` the Deployment + Service.
6. Wait a minute, then visit `https://<your-name>.alialjaffer.com` – real padlock, real name, and `served by pod: badge-xxxxx`.

⇩ `badge-deployment.yaml` ⇩ `front-door.yaml`

- [ ] Bare Pod ran and did **not** come back after deletion · badge live at your own real subdomain with a genuine TLS certificate · badge page shows the actual serving Pod's name.

SESSION 1 · DONE

You drove a real cluster from your terminal and named it yourself

Typed, not clicked

`kubectl run`, `create deployment`, `expose` – the same objects as Day 4, now from the CLI.

Annotations, not labels

One `external-dns` annotation gave your badge a real subdomain and a real TLS certificate.

After the break

Your badge is pinned at one replica. Next you'll let the cluster scale it by itself – then load-test the whole class at once.

🎮 Kahoot: Day 5 · Session 1 – kubectl verbs, Pods vs Deployments, and annotations vs labels.

Your badge is live at its own name. Now scale it – on purpose, in front of everyone.

Scale it by hand first, then hand the job to an autoscaler, then tune how fast it reacts in both directions. At the end, the whole class hits every subdomain at once to see whose autoscaler holds up.

FOUNDATIONS · YOU ALREADY DID THIS BY CLICKING

Manual scaling: you're the thermostat

```
kubectl scale deployment badge --replicas=5 -n <your-namespace>
kubectl get pods -n <your-namespace> --watch
```

This works

But **you** have to notice the traffic and type the command every time.

That gap is what HPA fills

A HorizontalPodAutoscaler watches usage and adjusts `replicas` for you.

HPA: self-healing, aimed at load instead of crashes

A HorizontalPodAutoscaler (HPA) watches a Deployment's resource usage and adjusts `replicas` for you.

```
kubectl autoscale deployment badge --cpu-percent=50 --min=1 --max=10 -n <your-namespace>
kubectl get hpa -n <your-namespace> --watch
```

One prerequisite

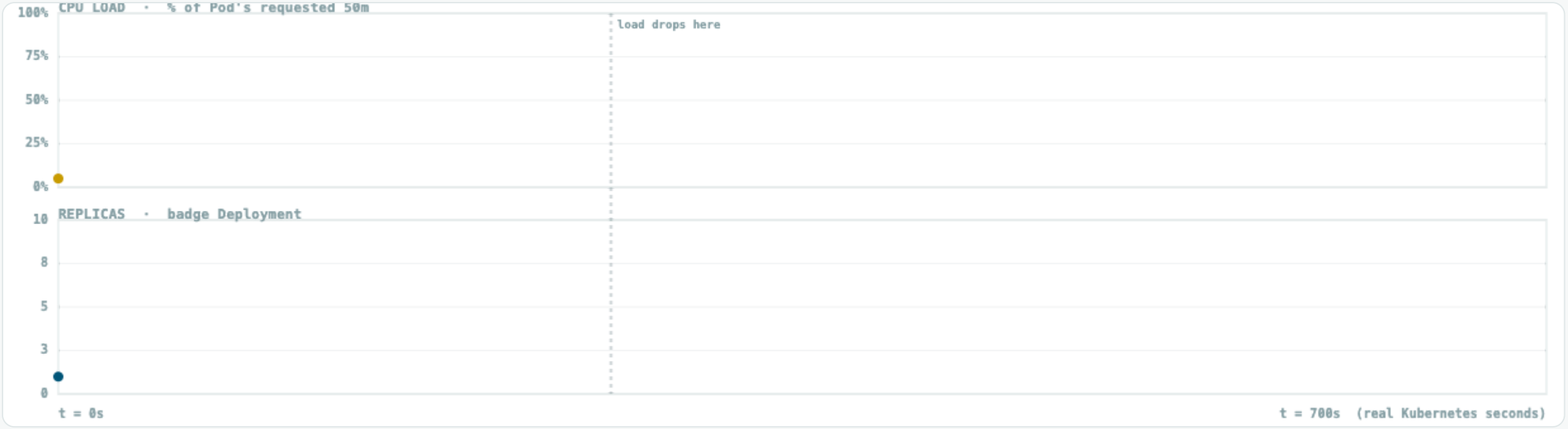
HPA measures usage as a percentage of what a Pod asked for. Milestone 9's Deployment already set this to 50m – confirm it: `kubectl set resources deployment badge --requests=cpu=50m`.

Set artificially low, on purpose

Makes the autoscaler easy to trigger live in class. A production service sizes this from real measured usage, not a demo-friendly round number.

HPA • SCALE-UP IS FAST, SCALE-DOWN IS SLOW BY DEFAULT – FEEL THE ASYMMETRY

Tune how fast the autoscaler reacts, in both directions



scaleDown behavior | Default (300s cooldown) | Tuned (15s) | ▶ Play load spike | ↺ Reset

Default scaleDown window: **300s**. Press **Play load spike** – same traffic curve either way, watch how long **replicas** takes to come back down after load drops.

The `behavior` block: no cluster-admin access needed

```yaml
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies:
      - type: Percent
        value: 100
        periodSeconds: 15
  scaleDown:
    stabilizationWindowSeconds: 15
    policies:
      - type: Percent
        value: 50
        periodSeconds: 15
```

> By default, HPA scales up almost instantly but scales down slowly — a built-in 5-minute cooldown (`stabilizationWindowSeconds: 300`) so it doesn't yo-yo on every small blip. `kubectl autoscale` can't set this — it only takes `--min`/`--max`/`--cpu-percent`. To add `behavior`, you `apply` a full HPA YAML over the one you already created — same name, same object, updated spec.

hey: a load-testing tool, one command

In a Terminal (MobaXterm, Terminal) run the following:

```
hey -z 60s -c 50 https://<your-name>.alialjaffer.com/
```

> `-z 60s` = run for 60 seconds. `-c 50` = 50 requests in flight at once. Point it at the real subdomain you stood up last session — the request has to survive DNS, the front door, and the load-balanced backend, same as any real visitor.

🏆 MILESTONE 10 · THE CLASS-WIDE LOAD-TEST SHOWDOWN

Scale, tune, then hit it with the rest of the class at once

1. Reset baseline: scale to 1 replica, set the CPU request back to 50m.
2. Turn on autoscaling: `kubectl autoscale --cpu-percent=50 --min=1 --max=10`.
3. Tune it. Fill in the scale-down blank, `apply` your own `hpa.yaml` with the full `behavior` block.
4. Confirm `hey` works against your subdomain with a short warm-up run.
5. On your instructor's go: `hey -z 90s -c 100` against your subdomain, with everyone else at the same moment.
6. Watch it live in a second terminal: `kubectl get hpa --watch` and `kubectl top pods`.
7. Refresh your badge during and after – watch `served by pod:` change as you land on different replicas.

⇩ `hpa.yaml`

- [ ] HPA scaled up under load and back down within ~15-30s of load ending · watched `kubectl get hpa --watch` / `top pods` move live · saw the served-by pod name change on refresh.

DAY 5 · DONE – AND THE WEEK

You drove a real Kubernetes cluster from the terminal, end to end

This morning

kubectl, typed Deployments & Services, annotations – your badge live at its own real subdomain with a real certificate.

This afternoon

Manual scaling, HPA, a tuned `behavior` block, and a class-wide load test you watched scale live.

The whole week

Server → Docker → CI/CD → networking → Kubernetes → autoscaling. You came in never having launched a server.

🎮 Kahoot: Day 5 · Session 2 – manual scaling, HPA, scaling behavior, and a grand mix of the whole week. Bragging rights on the line. 🏆

IN PARTNERSHIP WITH عزم السعودية SAUDI AZM SCCC by stc

DAY 5 • CLOSING

What's next?